Documento de Testing

Sistema de Gestión de Clínica

11 de Noviembre de 2026



Grupo 4:

Distefano, Gian Luca

Giordano, Valentino

Proia, Agustín

Yanosky, Santiago

Índice

Índice 2
Introducción 4
1. Testing de la clase AsociadoDTO 5
Escenarios 5
Clases de Equivalencia 5
Casos de Prueba 6
Resultados de Pruebas 7
Asignaciones Válidas 7
Reemplazo de Valor Previo 9
Conclusiones 10
2. Testing de Integración (AsociadoDTO, DataAccessObject y Base de Datos) 11
Alcance y Estrategia de Prueba 11
A. Parámetro asociado (en guardar(Asociado) y actualizar(Asociado)) 11
B. Parámetro límite (en cargarConLimite(int limite)) 12
C. Parámetro id (en cargar(int id) y ejecutarConsulta(query, id)) 12
D. Parámetro dni (en eliminar(String dni) y ejecutarDelete(query, dni)) 12
Verificación de Arquitectura, Estado y Excepciones de SQL 12
Suite 1: DataAccessObjectTest 13
Casos de Prueba 13
Resultados de Pruebas 15
Operaciones de Alta 15
Operaciones de Lectura 15
Operaciones de Modificación 16
Operaciones de Eliminación 17
Mapeo de Entidad a DTO 18
Suite 2: BaseDeDatosTest 19
Casos de Prueba 19
Resultados de Pruebas 20
Suite 3: BaseDeDatosSingletonTest 22
Casos de Prueba 22
Resultados de Pruebas 22
Conclusiones 23
3. Testing de Integración (Ambulancia, State y RetornoAutomatico) 25
Alcance y Estrategia de Prueba 25
Escenarios 25
Clases de Equivalencia 26
Suite 1: TestAmbulancia 26
Casos de Prueba 26
Resultados de Pruebas 28
Suite 2: TestIntegracionAmbulancia 28
Clases de Equivalencia 28
Casos de Prueba 30
Resultados de Pruebas 32

Suite 3: AmbulanciaConcurrenciaTest **34**
Casos de Prueba 34
Resultados de Pruebas 34
Suite 4: MantenimientoConcurrenciaTest **35**
Casos de Prueba 35
Resultados de Pruebas 36
Suite 5: RetornoAutomaticoTest **37**
Casos de Prueba 37
Resultados de Pruebas 38
Conclusiones **39**
4. Apéndice **40**
4.1 Resultados AsociadoDTO **40**
4.2 Resultados DataAccessObject **40**
4.3 Resultados BaseDeDatos **41**
4.4 Resultados BaseDeDatosSingleton **42**
4.5 Resultados Ambulancia **42**
4.6 Resultados IntegracionAmbulancia **43**
4.7 Resultados ConcurrenciaAmbulancia **43**
4.8 Resultados MantenimientoConcurrencia **44**
4.9 Resultados RetornoAutomatico **44**

Introducción

En este trabajo práctico desarrollamos y ejecutamos la estrategia de testing sobre el sistema de gestión de la Clínica, tomando como base las reglas de negocio definidas en la Especificación de Requisitos de Software (SRS) y los contratos implementados en el código provisto. El objetivo principal es verificar que los distintos módulos cumplan con la especificación, detectar los defectos o inconsistencias presentes en la implementación original y proponer las correcciones necesarias en cada caso.

El informe se organiza en tres etapas que van desde pruebas aisladas hasta pruebas de integración y concurrencia:

1. **Testing Unitario (AsociadoDTO):** se evalúan los getters y setters de la clase de transferencia en completo aislamiento, diseñando los casos de prueba a partir de particiones de equivalencia y valores límite bajo el enfoque de Diseño por Contrato.
2. **Testing de Integración de Persistencia (AsociadoDTO, DataAccessObject y BaseDeDatos):** se prueba la integración real de la capa de acceso a datos contra un motor MySQL 8.0 corriendo en un contenedor Docker, verificando el manejo del patrón Singleton, la creación del esquema con sus 20 registros semilla, las operaciones CRUD y la respuesta ante excepciones SQL.
3. **Testing de Integración y Concurrencia (Ambulancia, State y RetornoAutomatico):** se valida la delegación de responsabilidades, la matriz de transición de estados, el acceso concurrente de asociados y operarios al monitor de la ambulancia y el ciclo de vida del hilo de retorno automático, combinando el uso de mocks mediante Mockito con pruebas de integración entre las clases reales.

1. Testing de la clase AsociadoDTO

Escenarios

Nro. de Escenario	Descripción
1	AsociadoDTO recién creado con el constructor por defecto: atributos String en null y numero en 0
2	AsociadoDTO con el atributo bajo prueba cargado previamente con un valor válido: nombre="Juan", apellido="Perez", dni="12345678", ciudad="Mar del Plata", calle="Independencia", numero=1234, telefono="2235555555"

Clases de Equivalencia

Condición de entrada	**ID Clase**	**Clases Válidas**	**Clases Inválidas**
nombre	1	String <> null	-
apellido	2	String <> null	-
dni	3	String <> null	-
ciudad	4	String <> null	-
calle	5	String <> null	-
numero	6	numero >= 0	-
telefono	7	String <> null	-

Las reglas de validación establecidas en la SRS (sección 3.1) para los atributos de AsociadoDTO se consideran precondiciones de los setters. Bajo este enfoque, garantizar que los parámetros cumplan la precondición es responsabilidad de quien invoca el método, no del método invocado.
Al no existir una salida esperada contra la cual comparar el resultado obtenido, no es posible diseñar casos de prueba para esas entradas. Por este motivo, la tabla de particiones solo contiene clases válidas, y los casos de prueba cubren exclusivamente valores que satisfacen las precondiciones.

Casos de Prueba

ID Caso de Prueba	Tipo de Clase	Valores Entrada	Salida Esperada	Clases de Prueba Cubiertas	Esc.
1	Correcta	nombre = “Juan”	Se asigna el nombre “Juan”	1	1
2	Correcta	apellido = “Perez”	Se asigna el apellido “Perez”	2	1
3	Correcta	dni = “12345678”	Se asigna el DNI “12345678”	3	1
4	Correcta	ciudad = “Mar del Plata”	Se asigna la ciudad “Mar del Plata”	4	1
5	Correcta	calle = “Independencia”	Se asigna la calle “Independencia”	5	1
6	Correcta	numero = 1234	Se asigna el numero 1234	6	1
7	Correcta (valor límite)	numero = 0	Se asigna el numero 0	6	1
8	Correcta (valor límite)	numero = 1	Se asigna el numero 1	6	1
9	Correcta	telefono = “2235555555”	Se asigna el telefono “2235555555”	7	1
10	Correcta	nombre = “Pedro”	Se reemplaza el nombre “Juan” por “Pedro”	1	2
11	Correcta	apellido = “Gomez”	Se reemplaza el apellido “Perez” por “Gomez”	2	2
12	Correcta	dni = “87654321”	Se reemplaza el dni “12345678” por “87654321”	3	2
13	Correcta	ciudad = “Balcarce”	Se reemplaza la ciudad “Mar del Plata” por “Balcarce”	4	2
14	Correcta	calle = “Colon”	Se reemplaza la calle	5	2

			“Independencia” por “Colon”		
15	Correcta	numero = 1000	Se reemplaza el numero 1234 por 1000	6	2
16	Correcta (valor límite)	numero = 0	Se reemplaza el numero 1234 por 0	6	2
17	Correcta	telefono = “2234444444”	Se reemplaza el telefono “2235555555” por “2234444444”	7	2

Resultados de Pruebas

Asignaciones Válidas

Partiendo de un AsociadoDTO recién creado (escenario 1), se verifica que los setters guarden los valores válidos correctamente y que los getters los devuelvan.

Test	ID Caso de Prueba asociadas	Resultado
testSetNombre	1	OK
testSetApellido	2	OK
testSetDni	3	OK
testSetCiudad	4	OK
testSetCalle	5	OK
testSetNumero	6	OK
testSetNumeroAceptaCero (valor límite)	7	Fallo
testSetNumeroAceptaUno (valor límite)	8	OK
testSetTelefono	9	OK

Todos los tests de este grupo están armados de la forma: preparar, ejecutar, verificar. Tomamos como ejemplo testSetNombre:

```java
@Test
void testSetNombre() {
    AsociadoDTO asociado = new AsociadoDTO();

    asociado.setNombre("Juan");

    assertEquals("Juan", asociado.getNombre());
}
```

1. Preparación: se crea un AsociadoDTO con el constructor por defecto, lo que corresponde al escenario 1.
2. Ejecución: se invoca el setter del atributo bajo prueba con un valor perteneciente a su clase válida.
3. Verificación: se recupera el valor mediante el getter correspondiente y se comprueba con assertEquals que coincida con el asignado.

Los demás tests del grupo mantienen la misma estructura y solo cambian el atributo, el par setter/getter invocado y el valor de entrada, según lo detallado en la batería de pruebas.

Fallo: testSetNumeroAceptaCero

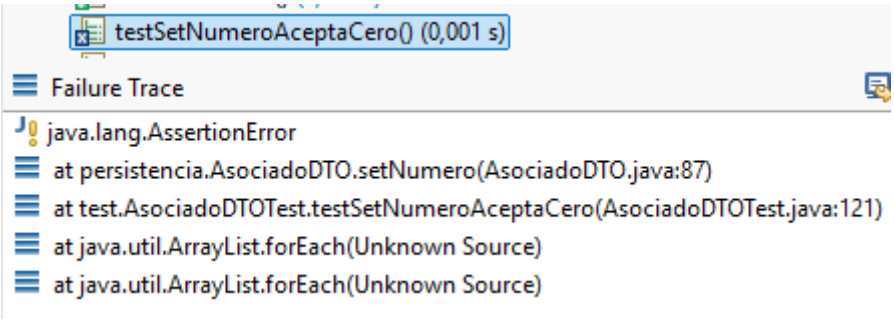
Captura del código del test:

```java
@Test
void testSetNumeroAceptaCero() {
    AsociadoDTO asociado = new AsociadoDTO();

    asociado.setNumero(0);

    assertEquals(0, asociado.getNumero());
}
```

El test corresponde al Escenario 1 y verifica el valor límite inferior de la clase válida 6 (numero >= 0). Partiendo de un AsociadoDTO recién creado, se asigna numero = 0 mediante el setter y se verifica que, mediante el getter, el valor se haya almacenado. Según la SRS, el número “debe ser mayor o igual a 0”, por lo que 0 cumple con la precondición y el setter debería aceptarlo. La salida esperada es que getNumero() devuelva 0.



La traza muestra que el error se produce en AsociadoDTO.setNumero en la línea 87, invocado desde la línea 121 del test, es decir, durante la llamada a setNumero(0). La ejecución nunca llega al assertEquals, esto indica que la falla no es una diferencia entre el valor esperado y el obtenido, sino que el propio setter rechaza el valor 0.

- Causa: la condición de la precondición implementada (numero > 0) no coincide con la especificada en la SRS (numero >= 0).
- Corrección propuesta: reemplazar assert numero > 0; por assert numero >= 0; en AsociadoDTO.setNumero.

- Observación: el defecto sólo se manifiesta al ejecutar con “-ea”. Sin esta opción, los assert no se evalúan.

Reemplazo de Valor Previo

Partiendo de un AsociadoDTO con el atributo bajo prueba cargado previamente con un valor válido (escenario 2), se verifica que los setters reemplacen ese valor por uno nuevo y que los getters devuelvan el valor actualizado.

Test	ID Caso de Prueba asociadas	Resultado
testSetNombreReemplazaValorPrevio	10	OK
testSetApellidoReemplazaValorPrevio	11	OK
testSetDniReemplazaValorPrevio	12	OK
testSetCiudadReemplazaValorPrevio	13	OK
testSetCalleReemplazaValorPrevio	14	OK
testSetNumeroReemplazaValorPrevio	15	OK
testSetNumeroCeroReemplazaValorPrevio	16	Fallo
testSetTelefonoReemplazaValorPrevio	17	OK

Los tests de este grupo siguen la misma estructura de preparar, ejecutar y verificar que el grupo anterior, con una diferencia en la preparación: además de crear el AsociadoDTO, se carga un valor válido inicial mediante el setter. Tomamos como ejemplo testSetNombreReemplazaValorPrevio:

```
@Test
void testSetNombreReemplazaValorPrevio() {
    AsociadoDTO asociado = new AsociadoDTO();
    asociado.setNombre("Juan");

    asociado.setNombre("Pedro");

    assertEquals("Pedro", asociado.getNombre());
}
```

Luego de la preparación, se asigna un segundo valor válido y se verifica con assertEquals que el getter devuelva el nuevo valor. Los demás tests del grupo mantienen la misma estructura y solo cambian el atributo, el par setter/getter invocado y los valores, según lo detallado en la batería de pruebas.

Fallo: testSetNumeroCeroReemplazaValorPrevio

Este test corresponde al escenario 2. Partiendo de un AsociadoDTO con numero = 1234, se asigna el valor límite 0 (clase válida 6) y se espera que reemplace al valor previo. El test falla con el mismo java.lang.AssertionError, lanzado en AsociadoDTO.setNumero (línea 87) durante la llamada setNumero(0). La causa es el mismo defecto descripto para testSetNumeroAceptaCero: el setter verifica numero > 0 en lugar de numero >= 0, como

establece la SRS. Este caso muestra que el defecto se manifiesta independientemente del estado previo del objeto, y que, al rechazarse la asignación, el atributo conserva el valor 1234 en lugar del 0 esperado.

Conclusiones

Se diseñó y ejecutó una suite de 17 casos de prueba unitarios sobre la clase AsociadoDTO, construidos a partir de las reglas de la SRS (sección 3.1) mediante la técnica de particiones de equivalencia y análisis de valores límite. Los casos se organizaron en dos escenarios: la asignación de valores sobre un objeto recién creado y el reemplazo de un valor previamente cargado. Las pruebas se ejecutaron de forma completamente aislada, sin depender de servicios externos, tal como lo requiere la consigna.

De los 17 casos ejecutados, 15 resultaron exitosos y 2 fallaron. Ambos fallos corresponden al valor límite numero = 0 y responden a un único defecto: el setter setNumero verifica la precondición numero > 0, mientras que la SRS establece numero >= 0. Es decir, la implementación exige una precondición más estricta que la especificada y rechaza un valor válido del dominio. El hecho de que el defecto se manifieste en ambos escenarios demuestra que es independiente del estado previo del objeto. Este resultado confirma la utilidad del análisis de valores límite: un valor representativo de la clase válida, como 1234, no habría permitido detectarlo, ya que el error se encuentra exactamente en el borde del dominio.
Por otro lado, se observó que las precondiciones de la clase están implementadas mediante assert, cuya evaluación depende de que la JVM se ejecute con la opción -ea. Sin esta opción las verificaciones no se realizan, el valor 0 se almacena sin inconvenientes y los dos casos que detectan el defecto pasan exitosamente, lo que oculta el error. Esto constituye un riesgo, ya que la detección del defecto depende de la configuración de ejecución y no del código en sí.

2. Testing de Integración (AsociadoDTO, DataAccessObject y Base de Datos)

Alcance y Estrategia de Prueba

El objetivo de esta etapa es verificar la integración real entre la capa de acceso a datos (DataAccessObject), el administrador de conexión (BaseDeDatos) y los objetos de transferencia (AsociadoDTO).

Para garantizar la independencia y repetibilidad de las pruebas (evitando que un alta, baja o modificación afecte el resultado de la prueba siguiente), se configuró un fixture con @BeforeEach que invoca BaseDeDatos.getInstancia().iniciarBD() antes de cada caso de prueba, restableciendo la tabla Asociados con sus 20 registros semilla. Por su parte, las pruebas destructivas sobre el ciclo de vida del Singleton (cerrarConexion) se aislaron en la suite BaseDeDatosSingletonTest ordenadas mediante @Order.

A. Parámetro asociado (en guardar(Asociado) y actualizar(Asociado))

Condición entrada	ID Clase	Clases Válidas	ID Clase	Clases Inválidas
Instancia Asociado	1	Objeto Asociado no nulo con campos completos.	-	-
Unicidad de dni (en guardar)	2	dni no existente en la tabla Asociados.	4	dni ya registrado en la BD (viola UNIQUE).
Existencia de dni (en actualizar)	3	dni perteneciente a un asociado existente en la BD.	-	-

B. Parámetro límite (en cargarConLimite(int limite))

Condición entrada	ID Clase	Clases Válidas	ID Clase	Clases Inválidas
limite	5.1	1 <= limite <= 20	-	-
	5.2	limite > 20 (Mayor al total de registros en la BD)	-	-

C. Parámetro id (en cargar(int id) y ejecutarConsulta(query, id))

Condición entrada	ID Clase	Clases Válidas	ID Clase	Clases Inválidas
id	6.1	1 <= id <= 20 (ID existente en la BD)	-	-
	6.2	id > 20 (ID inexistente en la BD, retorna null)		

D. Parámetro dni (en eliminar(String dni) y ejecutarDelete(query, dni))

Condición entrada	ID Clase	Clases Válidas	ID Clase	Clases Inválidas
dni	7.1	dni no nulo ni vacío existente en la BD.	-	-
	7.2	dni no nulo ni vacío inexistente en la BD (no altera filas).	-	-

Al igual que en la etapa anterior, las condiciones “null”, límite positivo e identificador positivo de los métodos del DAO se consideran precondiciones, garantizar su cumplimiento es responsabilidad de quien invoca el método. Al no existir una salida esperada para entradas que violan una precondición, no se diseñan casos de prueba para ellas. La única clase inválida es la de DNI duplicado.

Verificación de Arquitectura, Estado y Excepciones de SQL

Además del particionamiento de datos de entrada sobre el DAO, la suite de integración evalúa requisitos arquitectónicos y de robustez según la SRS:

- Patrón Singleton y Ciclo de Vida: Se comprueba que BaseDeDatos.getInstancia() mantenga una única referencia activa y se evalúa el comportamiento del sistema ante la invocación de cerrarConexion().
- Inicialización del esquema y datos semilla: Se verifica la ejecución de iniciarBD() (creación de la tabla Asociados y consistencia de las 20 tuplas iniciales).
- Robustez frente a excepciones SQL: Se evalúa la respuesta del motor ante violaciones de restricciones de integridad (UNIQUE en DNI) y sentencias SQL inválidas.

Suite 1: DataAccessObjectTest

Nro. de Escenario	Descripción
1	Base de datos inicializada mediante iniciarBD(), con la tabla Asociados creada y cargada con sus 20 registros semilla.

Casos de Prueba

ID Caso de Prueba	Tipo de Clase	Valores Entrada	Salida Esperada	Clases de Prueba Cubiertas	Esc.
1	Correcta	asociado = ("Valentino", "Giordano", "46632600", "2235964847", "Enrique Bochini", 1850, "Mar del Plata")	Se guarda el asociado; el total pasa a 21 tuplas.	1, 2	1
2	Incorrecta	dni = "35521599" (insertado 2 veces para Agustín Proia)	Se rechaza el alta lanzando SQLException por violación de clave UNIQUE.	4	1
3	Correcta	Invocación a dao.cargarBD()	Se recupera la lista con los 20 asociados semilla.	-	1
4	Correcta	limite = 1	Se recupera una lista con 1 asociado (Juan State).	5.1	1

5	Correcta	limite = 5	Se recupera una lista con exactamente 5 asociados.	5.1	1
6	Correcta	limite = 50	Se recuperan los 20 asociados existentes sin errores.	5.2	1
7	Correcta	id = 1	Se recupera el DTO de Juan State (id = 1).	6.1	1
8	Correcta	id = 999	Retorna null al no existir tupla con ese ID.	6.2	1
9	Correcta	asociado = ("MariaModificada", "TemplateModificada", "87654321", "2238888888", "Calle Modificada", 456, "Mar del Plata")	Se actualizan los datos de María (id = 2) en la BD.	1, 3	1
10	Correcta	dni = "12345678" (Juan State)	Se elimina el asociado; el total pasa a 19 tuplas.	7.1	1
11	Correcta	dni = "99999999"	No se altera la base de datos; se mantienen 20 tuplas.	7.2	1
12	Correcta	asociado = ("Santiago", "Yanosky", "35111222", "2235559988", "Alberti", 500, "Mar del Plata")	DTO con los atributos de dominio alineados.	1	1
13	Correcta	id = 99, "Ana", "Gomez", "22334455", "Colon", 500, "Mar del Plata", "2234445566"	DTO conserva los datos ingresados.	-	1

Resultados de Pruebas

Operaciones de Alta

Partiendo de la base de datos restablecida con sus 20 registros semilla (escenario 1), se verifica el comportamiento del método guardar al insertar entidades nuevas y ante la duplicación de identificadores únicos.

Test	ID Caso de Prueba asociadas	Resultado
testGuardarNuevoAsociadoExitoso	1	OK
testGuardarDniDuplicadoLanzaSQLException	2	OK

A modo de ejemplo representativo, se expone la implementación de la prueba de integridad que verifica la captura de excepciones SQL ante claves duplicadas:

```
@Test
void testGuardarDniDuplicadoLanzaSQLException() throws SQLException {
    Domicilio domicilio = new Domicilio("Dorrego", 3100, "Mar del Plata");
    Asociado duplicado = new Asociado("Agustin", "Proia", "35521599", "2231234567", domicilio);

    // Guardamos por primera vez al asociado
    dao.guardar(duplicado);

    // Intentar guardarlo por segunda vez con el mismo DNI debe disparar SQLException
    assertThrows(SQLException.class, () -> {
        dao.guardar(duplicado);
    }, "Guardar un asociado con DNI repetido debe lanzar SQLException por restricción UNIQUE.");
}
```

Operaciones de Lectura

Se evalúa la recuperación de registros desde la base de datos a través de consultas generales (cargarBD), consultas parametrizadas con límites (cargarConLimite) y búsquedas por clave primaria (cargar(id)).

Test	ID Caso de Prueba asociadas	Resultado
testCargarBDRecuperaTodosLosRegistrosSemilla	3	OK
testCargarConLimiteUno	4	OK
testCargarConLimiteCinco	5	OK

testCargarConLimiteMayorAlTotal	6	OK
testCargarPorId	7	Fallo
testCargarPorIdInexistente	8	Fallo

A modo de ejemplo representativo, se expone la implementación de la prueba funcional destinada a recuperar un asociado específico existente en la base de datos:

```
@Test
void testCargarPorId() throws SQLException {
    // Pedimos cargar el asociado con id = 1 (Juan State)
    AsociadoDTO dto = dao.cargar(1);

    assertNotNull(dto);
    assertEquals("Juan", dto.getNombre());
}
```

Fallo: testCargarPorId() y testCargarPorIdInexistente()

En ambos métodos la suite arrojó un **Fallo** debido al disparo de una excepción no controlada de tipo java.sql.SQLException: Before start of result set.
En la implementación de DataAccessObject.cargar(int id), se omitió llamar al método de avance resultado.next(), procediendo a leer directamente los atributos mediante resultado.getInt("id") y resultado.getString("nombre") sobre un cursor no posicionado. Esto lo que generaba era principalmente lo siguiente:

- Para un Id existente el sistema es incapaz de leer el registro que devolvió la base de datos, y termina abortando mediante una SQLException.
- Para un Id inexistente en vez de evaluar el resultado.next() mediante un if para devolver null, el método colapsa por la misma razón que en el caso anterior y aborta mediante una SQLException.
- **Causa:** se omitió llamar al método de avance resultado.next(), procediendo a leer directamente los atributos mediante resultado.getInt("id") y resultado.getString("nombre") sobre un cursor no posicionado.
- **Corrección propuesta:** utilizar un if(resultado.next()) para validar que la consulta devuelve una fila del asociado al cual le corresponde ese id.

Operaciones de Modificación

Se comprueba la actualización de los datos de un asociado existente en la base de datos mediante el método actualizar.

Test	ID Caso de Prueba asociadas	Resultado
testDemostrarInconsistenciaEnActualizar	9	Fallo

```java
@Test
void testDemostrarInconsistenciaEnActualizar() throws SQLException {
    // Datos correspondientes a Maria Template (DNI 87654321, id = 2) modificados
    Domicilio nuevoDom = new Domicilio("Calle Modificada", 456, "Mar del Plata");
    Asociado mariaEditada = new Asociado("MariaModificada", "TemplateModificada", "87654321", "2238888888", nuevoDom);

    dao.actualizar(mariaEditada);

    // Segun la SRS, la actualizacion se realiza por DNI: debe modificarse Maria y Juan debe quedar intacto
    ArrayList<AsociadoDTO> lista = dao.cargarBD();
    AsociadoDTO maria = null;
    AsociadoDTO juan = null;
    for (AsociadoDTO dto : lista) {
        if ("87654321".equals(dto.getDni())) {
            maria = dto;
        } else if ("12345678".equals(dto.getDni())) {
            juan = dto;
        }
    }

    assertEquals("MariaModificada", maria.getNombre(), "Los datos de Maria deben haberse actualizado.");

    assertEquals("Juan", juan.getNombre(), "Juan State no debe haber sido modificado.");
}
```

Fallo: testDemostrarInconsistenciaEnActualizar()

Al ejecutar este caso de prueba, la suite arrojó un **Fallo** debido al disparo de una excepción no controlada de tipo java.sql.SQLIntegrityConstraintViolationException: Duplicate entry '87654321' for key 'Asociados.DNI'.

El origen del defecto radica en la delegación de la operación: DataAccessObject.actualizar() invoca a BaseDeDatos.ejecutarActualizacion(), cuya sentencia SQL contiene la cláusula WHERE id = ?. Sin embargo, en la línea 118 de la clase BaseDeDatos, el parámetro del identificador se encuentra **hardcodeado** en el valor 1 (stmt.setInt(8, 1)), incumpliendo lo establecido en la SRS, que especifica que las actualizaciones deben realizarse tomando como criterio único el DNI.

Como consecuencia de este hardcodeo, al intentar modificar los datos de María Template (DNI '87654321', correspondiente al registro id = 2), el sistema intentó aplicar dichos cambios sobre la fila con id = 1 (asociada originalmente a Juan State). El motor MySQL rechazó inmediatamente la transacción, ya que el DNI provisto ya existía en otra fila de la tabla, violando la restricción de unicidad (UNIQUE).

- **Corrección propuesta:** Quitar la línea 118 del método ejecutarActualizacion() de la clase BaseDeDatos, evitando así que el motor de la BD intente siempre actualizar el registro asociado al id=1.

Operaciones de Eliminación

Partiendo de la base de datos inicializada con sus 20 registros semilla (escenario 1), se valida la baja física de registros en la tabla Asociados mediante el método dao.eliminar(String dni), comprobando la eliminación de tuplas existentes y la ausencia de efectos ante identificadores inexistentes.

Test	ID Caso de Prueba asociadas	Resultado
testEliminarAsociadoExistente	10	OK

testEliminarDniInexistenteNoAlteraRegistros	11	OK

A modo de ejemplo dejamos el código de la prueba funcional que verifica la correcta eliminación de un registro existente.

```
@Test
// A traves del DAO deberiamos poder borrar a un asociado
void testEliminarAsociadoExistente() throws SQLException {
    String dniABorrar = "12345678"; // DNI de Juan State

    dao.eliminar(dniABorrar);

    // Verificar que el tamaño de la base bajó a 19
    ArrayList<AsociadoDTO> lista = dao.cargarBD();
    assertEquals(19, lista.size(), "Luego de eliminar a un asociado, la lista debe contener 19 registros.");

    // Verificar que el asociado borrado ya no se encuentra en la colección
    boolean encontrado = false;
    for (AsociadoDTO dto : lista) {
        if (dniABorrar.equals(dto.getDni())) {
            encontrado = true;
            break;
        }
    }
    assertFalse(encontrado, "El asociado eliminado no debe figurar en la base de datos.");
}
```

Mapeo de Entidad a DTO

Se evalúa la transformación entre las entidades del modelo de dominio (Asociado) y los objetos de transferencia de datos (AsociadoDTO), verificando que los atributos se trasladen correctamente y que el identificador recibido por CrearDTO no se conserve en el DTO.

Test	ID Caso de Prueba asociadas	Resultado
testCrearDTOAlineaAtributosDeDominio	12	OK
testNoSeGuardaIdEnCrearDTO	13	OK

A modo de ejemplo representativo, se expone la prueba que verifica que los atributos del Asociado se trasladen correctamente al DTO:

```
@Test
void testCrearDTOAlineaAtributosDeDominio() {
    Domicilio dom = new Domicilio("Alberti", 500, "Mar del Plata");
    Asociado asociado = new Asociado("Santiago", "Yanosky", "35111222", "2235559988", dom);

    AsociadoDTO dto = dao.CrearDTO(asociado);

    assertNotNull(dto);
    assertEquals("Santiago", dto.getNombre());
    assertEquals("Yanosky", dto.getApellido());
    assertEquals("35111222", dto.getDni());
    assertEquals("Alberti", dto.getCalle());
    assertEquals(500, dto.getNumero());
    assertEquals("Mar del Plata", dto.getCiudad());
    assertEquals("2235559988", dto.getTelefono());
}
```

Inconsistencia: Parámetro sobrante en CrearDTO()

Al mirar el método CrearDTO en el DAO, vemos que recibe el id como primer parámetro, pero dentro del código no se usa para nada.

Revisando la SRS (sección 3.1), queda claro que AsociadoDTO no debe llevar ID, ya que solo define los datos personales y usa el DNI como único identificador. Por lo tanto, que CrearDTO reciba un id es un error de diseño en el DAO: está pidiendo un parámetro que no corresponde con la especificación de la SRS y que encima se termina descartando.

Suite 2: BaseDeDatosTest

Nro. de Escenario	Descripción
1	Base de datos inicializada mediante iniciarBD(), con la tabla Asociados creada y cargada con sus 20 registros semilla.

Casos de Prueba

ID Caso de Prueba	Tipo de Clase	Valores Entrada	Salida Esperada	Clases de Prueba Cubiertas	Esc.
14	Correcta	Invocación a iniciarBD()	Se crea la tabla Asociados en el esquema Grupo_7.	-	1
15	Correcta	SELECT COUNT(*) AS total FROM Asociados;	Total de tuplas semilla igual a 20.	-	1
16	Correcta	SELECT * FROM Asociados WHERE id = 1;	Registro id = 1 contiene los datos de Juan State.	7.1	1
17	Correcta	AsociadoDTO("Lionel", "Messi", "10101010", "Rosario", "Lavalle", 1010, "3415555555")	Se inserta la fila; el total pasa a 21 tuplas.	1, 2	1

18	Correcta	SELECT * FROM Asociados; con id = -1	Retorna las 20 tuplas completas de la base de datos.	-	1
19	Correcta	SELECT * FROM Asociados WHERE id = ? ; con id = 2	Retorna el registro de María Template.	7.1	1
20	Correcta	SELECT * FROM Asociados WHERE id = ? ; con id = 9999	Retorna cursor vacío (rs.next() == false).	7.2	1
21	Correcta	AsociadoDTO("JuanActualizado", "StateActualizado", "12345678", "Mar del Plata", "San Juan", 500, "2234441111")	Se modifican los datos en la tupla con id = 1.	1, 3	1
22	Correcta	DTO con DNI "99999999" y query con WHERE DNI = '00000000'	Ninguna tupla es modificada en la base.	5	1
23	Correcta	dni = "12345678"	Tupla con id 1 eliminada; total baja a 19 tuplas.	8.1	1
24	Correcta	dni = "99999999"	Total de registros se mantiene en 20 tuplas.	8.2	1
25	Incorrecta	Inserción directa de DTO con DNI "12345678" repetido	Lanza SQLException por violación de clave UNIQUE.	4	1
26	Incorrecta	SELECT * FROM TablaQueNoExiste; con id = -1	Lanza SQLException por tabla inexistente.	-	1

Resultados de Pruebas

En esta suite probamos directamente los métodos de la clase BaseDeDatos contra el contenedor de MySQL en Docker, validando que la inicialización cree la tabla y cargue la semilla, que las consultas directas (ejecutarInsert, ejecutarConsulta, ejecutarActualizacion, ejecutarDelete) impacten en la base y que se capturen las excepciones de SQL.

Test	ID Caso de Prueba asociadas	Resultado
testCreacionTablaAsociados	14	OK
testDatosSemillaCantidadExacta	15	OK
testDatosSemillaContenidoPrimerRegistro	16	OK
testCRUD_Create_ejecutarInsert	17	OK
testCRUD_Read_ejecutarConsultaSinParametro	18	OK
testCRUD_Read_ejecutarConsultaConParametroId	19	OK
testCRUD_Read_idInexistenteRetornaVacio	20	OK
testCRUD_Update_ejecutarActualizacion	21	OK
testCRUD_Update_asociadoInexistenteNoModificaRegistros	22	OK
testCRUD_Delete_ejecutarDelete	23	OK
testCRUD_Delete_dniInexistenteNoAlteraRegistros	24	OK
testExcepcionPorDniDuplicado	25	OK
testExcepcionPorConsultaSqlInvalida	26	OK

A modo de ejemplo representativo, mostramos la prueba donde verificamos que la base de datos tire una SQLException al intentar insertar un DNI duplicado:

```
@Test
void testExcepcionPorDniDuplicado() {
    // El DNI 12345678 ya existe en la semilla
    AsociadoDTO duplicado = new AsociadoDTO("Otro", "Usuario", "12345678", "Mar del Plata", "Belgrano", 100, "2230000000");
    String insertQuery = "INSERT INTO Asociados (nombre, apellido, DNI, calle, numero, ciudad, telefono) VALUES (?, ?, ?, ?, ?, ?, ?);";

    assertThrows(SQLException.class, () -> {
        bd.ejecutarInsert(insertQuery, duplicado);
    }, "Debe lanzar SQLException por violar la restricción UNIQUE del campo DNI.");
}
```

Suite 3: BaseDeDatosSingletonTest

Nro. de Escenario	Descripción
1	Base de datos inicializada mediante iniciarBD(), con la tabla Asociados creada y cargada con sus 20 registros semilla.

Casos de Prueba

Tal como se aclaró en la estrategia de prueba, los tests que evalúan el patrón Singleton y el cierre de la conexión se separaron en una suite propia para no contaminar al resto de las pruebas.

ID Caso de Prueba	Tipo de Clase	Valores Entrada	Salida Esperada	Clases de Prueba Cubiertas	Esc.
27	Correcta	Dos llamadas consecutivas a BaseDeDatos.getInstancia()	Retorna exactamente la misma referencia en memoria (assertSame).	-	1
28	Correcta	Invocación a bd.getConexion().isValid(2)	Retorna true (conexión abierta y respondiendo activamente a MySQL).	-	1
29	Correcta	Invocación a cerrarConexion() y posterior llamada a getInstancia()	Retorna una conexión activa y no nula para poder seguir trabajando.	-	1

Resultados de Pruebas

Test	ID Caso de Prueba asociadas	Resultado
testPatronSingletonRetornaMismaInstancia	27	OK
testConexionActivaYValida	28	OK
testSiCierroLaConexionPuedoVolverAIniciarla	29	Fallo

A modo de ejemplo dejamos a continuacion el metodo donde verificamos que si cerramos la conexion deberiamos poder volver a abrirla al simplemente obtener una nueva instancia de la base de datos:

```
@Test
@Order(3)
void testSiCierroLaConexionPuedoVolverAIniciarla() throws SQLException {
    BaseDeDatos bd = BaseDeDatos.getInstancia();
    bd.cerrarConexion();

    BaseDeDatos bd_2 = BaseDeDatos.getInstancia();

    Connection conexion = bd_2.getConexion();
    assertNotNull(conexion, "La conexion con la base de datos no debe ser nula");
    assertFalse(conexion.isClosed(), "La conexión debe encontrarse abierta tras instanciar la clase.");
}
```

Fallo: testSiCierroLaConexionPuedoVolverAIniciarla()

En esta prueba buscamos verificar el ciclo de vida completo de la conexión: primero obtenemos la instancia única del Singleton, cerramos su conexión llamando a cerrarConexion(), y enseguida volvemos a pedir la instancia con BaseDeDatos.getInstancia() (simulando que otra parte del sistema o un nuevo DataAccessObject necesita volver a usar la base de datos). La salida esperada es que getInstancia() nos devuelva una conexión abierta y lista para usar (conexion.isClosed() == false).
Al ejecutar el test, la suite arrojó un **Fallo** por incumplimiento de aserción (org.opentest4j.AssertionFailedError: La conexión debe encontrarse abierta tras instanciar la clase. ==> expected: <false> but was: <true>) en la línea 53 del test, justo en el assertFalse(conexion.isClosed()).

- **Corrección propuesta:** Al llamar al método cerrarConexion() de la clase BaseDeDatos, no solo se debería cerrar la conexión, sino que también debería setear a la instancia de la base de datos en null, asi si se quisiera volver a instanciar a la base de datos se crearía una nueva instancia con una conexión abierta.

Conclusiones

Se diseñaron y ejecutaron 3 suites de pruebas de integración (DataAccessObjectTest, BaseDeDatosTest y BaseDeDatosSingletonTest) que abarcan un total de 29 casos de prueba sobre los módulos AsociadoDTO, DataAccessObject y BaseDeDatos, construidos a partir de las reglas de la SRS (secciones 3.1, 3.2 y 3.3) mediante particiones de equivalencia y valores límite. A diferencia del punto anterior, estas pruebas se ejecutaron integrando los componentes contra un motor MySQL 8.0 real corriendo en un contenedor Docker, restableciendo la tabla Asociados con sus 20 registros semilla antes de cada prueba mediante @BeforeEach y aislando las pruebas destructivas del Singleton en una suite independiente.

De la ejecución de las pruebas se lograron detectar múltiples defectos funcionales y de diseño que impedían el correcto funcionamiento de la capa de persistencia:

- **Manejo del cursor JDBC en cargar(int id):** La omisión del llamado a resultado.next() provocaba que el método abortara siempre con una SQLException (*Before start of result set*), tanto para IDs existentes como para IDs inexistentes (donde debía retornar null).
- **Actualización hardcodeada:** En BaseDeDatos.ejecutarActualizacion() se fijó el octavo parámetro en 1 (stmt.setInt(8, 1)) sobre una cláusula WHERE id = ?, incumpliendo la SRS que exige actualizar por DNI. Esto provocaba un error de clave duplicada (UNIQUE) al intentar modificar cualquier otro asociado existente, o sobrescribía silenciosamente al registro id = 1 si se ingresaba un DNI nuevo.
- **Inconsistencia en el mapeo y ciclo de vida del Singleton:** El método CrearDTO en el DAO recibía un parámetro id que descartaba por completo (ya que según la SRS AsociadoDTO se identifica por DNI y no lleva ID), y el método cerrarConexion() de BaseDeDatos cerraba la conexión física pero no reiniciaba instancia = null, dejando al Singleton inutilizable ante futuras llamadas a getInstancia().

Estos resultados demuestran la importancia de realizar pruebas de integración contra una base de datos real: defectos críticos como la falta de avance del cursor en el ResultSet, el choque de restricciones UNIQUE en el motor SQL por el hardcodeo del UPDATE, o la pérdida de conexión activa en el Singleton no habrían podido detectarse mediante pruebas unitarias aisladas ni simulando la base de datos.

3. Testing de Integración (Ambulancia, State y RetornoAutomatico)

Alcance y Estrategia de Prueba

El objetivo de esta etapa es verificar la integración entre la clase Ambulancia, el patrón **State** que modela su ciclo de vida y el hilo RetornoAutomatico, comprobando tres aspectos:

- **Delegación de responsabilidades:** que Ambulancia derive cada solicitud al objeto State vigente y no implemente la lógica de transición por su cuenta.
- **Corrección de la máquina de estados:** que cada evento (SolicitudDeTraslado, SolicitudDeAtencionDomicilio, SolicitudMantenimiento y RetornoClinica) produzca la transición esperada o sea rechazado sin alterar el estado.
- **Concurrencia y sincronización:** que los hilos que compiten por la ambulancia esperen correctamente, que la liberación con notifyAll() reactive a los waiters y que el retorno automático no se solape con las operaciones protegidas por synchronized.

La estrategia combina pruebas unitarias y de integración:

- Se utilizan objetos reales de Ambulancia en todos los casos, para no perder el comportamiento de sincronización ni el patrón State.
- Se emplean **mocks de Mockito** para Asociado, Operario y State, de modo de aislar la unidad bajo prueba y controlar los valores de retorno y las invocaciones.
- La sincronización de los tests se realiza con CountDownLatch, Semaphore, AtomicInteger y Thread.State, evitando Thread.sleep() como mecanismo de sincronización. La única espera temporal inevitable es la del intervalo fijo de RetornoAutomatico (10 segundos), por lo que esos casos utilizan un margen de 3 segundos.
- Cada test parte de una instancia nueva de Ambulancia, por lo que no hay estado compartido entre casos.

Escenarios

Nro. de Escenario	Descripción
1	Ambulancia recién creada con el constructor por defecto: estadoActual = DisponibleState, ocupado = false, isSimulacionActiva = true y lista de asociados vacía.
2	Ambulancia forzada a cada estado concreto del patrón State (TrasladandoPacienteClinica, AtendiendoEnDomicilio, EnTaller, RegresandoDelTaller y RegresandoClinicaSinPaciente).

3	Ambulancia con el estado actual sustituido por un mock de State, para aislar la delegación de responsabilidades de Ambulancia.
4	Ambulancia en estado *TrasladandoPacienteClinica* (ocupado = true) con dos asociados concurrentes esperando turno.
5	Ambulancia en estado *TrasladandoPacienteClinica* (ocupado = true) con un operario esperando para solicitar mantenimiento.
6	Ambulancia con RetornoAutomatico en ejecución y simulación activa.
7	Ambulancia con la simulación finalizada (isSimulacionActiva = false).

Clases de Equivalencia

Dado que los métodos bajo prueba no reciben parámetros, las condiciones de entrada de esta etapa no son argumentos, sino el estado del sistema y el evento que recibe la ambulancia. Por lo tanto, las clases de equivalencia se definen para cada suite según corresponda:

- TestIntegracionAmbulancia: la condición de entrada es el estado actual de la ambulancia, y las clases corresponden a los eventos que puede recibir en ese estado (ver tabla en la sección de la suite).
- TestAmbulancia, AmbulanciaConcurrenciaTest, MantenimientoConcurrenciaTest y RetornoAutomaticoTest: las condiciones de entrada (estado sustituido por un mock, ambulancia ocupada, simulación activa o finalizada) quedan completamente determinadas por los escenarios 3 a 7, por lo que los casos de prueba se diseñan a partir de dichos escenarios y no se definen tablas de particiones adicionales.

Suite 1: TestAmbulancia

Casos de Prueba

ID Caso de Prueba	Tipo de Clase	Valores Entrada	Salida Esperada	Clases de Prueba Cubiertas	Esc.
1	Correcta	ambulancia.setState(mock(State)); trasladarALaClinica(asociadoMock)	Se invoca exactamente una vez SolicitudDeTraslado() sobre el estado y se espera que esté	-	3

			ocupada la ambulancia.		
2	Correcta	ambulancia.setState(mock(State)); retornoAutomatico(mock(RetornoAutomatico))	Se invoca exactamente una vez RetornoClinica() sobre el estado.	-	3
3	Correcta	ambulancia.setState(mock(State)); solicitarMantenimiento(operarioMock)	Se invoca exactamente una vez SolicitudMantenimiento() sobre el estado y se espera que esté ocupada la ambulancia.	-	3
4	Correcta	ambulancia.setState(mock(State)); atenderDomicilio(asociadoMock)	Se invoca exactamente una vez SolicitudDeAtencionDomicilio() sobre el estado y se espera que esté ocupada la ambulancia.	-	3
5	Correcta	ambulancia.setState(mock(State)); ambulancia.setOcupado(true) regresarSinPaciente()	Se invoca exactamente una vez RetornoClinica() sobre el estado y se espera que no esté ocupada la ambulancia.	-	3
6	Correcta	ambulancia.setState(mock(State)); ambulancia.setOcupado(true) volviendoDelTaller()	Se invoca exactamente una vez SolicitudMantenimiento() sobre el estado y se espera que no esté ocupada la ambulancia.	-	3
7	Correcta	ambulancia.setState(estadoActualMock); con doAnswer en estadoActualMock.Solici	El estado actual de la ambulancia pasa a ser estadoSiguienteMock	-	3

		tudDeTraslado() que hace ambulancia.setState(estadoSiguienteMock); ambulancia.trasladarALaClinica(asociadoMock)			

Resultados de Pruebas

Test	ID Caso de Prueba asociadas	Resultado
debeDelegarTrasladoAlEstadoActual	1	OK
debeDelegarRetornoAlEstadoActual	2	OK
debeDelegarMantenimientoAlEstadoActual	3	OK
debeDelegarAtencionDomicilioAlEstadoActual	4	OK
debeDelegarRegresarSinPacienteYLiberarAmbulancia	5	OK
debeDelegarVolviendoDelTallerYLiberarAmbulancia	6	OK
debePermitirAlEstadoSimularTransicion	7	OK

A modo de ejemplo, se deja adjunto uno de los test realizados:

```
@Test
void debeDelegarTrasladoAlEstadoActual() {
     Ambulancia ambulancia = new Ambulancia();
     State estadoMock = mock(State.class);
     Asociado asociadoMock = mock(Asociado.class);

     ambulancia.setState(estadoMock);

     ambulancia.trasladarALaClinica(asociadoMock);

     verify(estadoMock, times(1)).SolicitudDeTraslado();
     verifyNoMoreInteractions(estadoMock);
     assertTrue(ambulancia.isOcupado());
}
```

Suite 2: TestIntegracionAmbulancia

Clases de Equivalencia

Se tomó como condición de entrada el estado actual de la ambulancia, y como clases los eventos que puede recibir en ese estado. Dado que cada estado implementa los cuatro métodos de la interfaz State, todos los eventos tienen un comportamiento definido en todos los estados: algunos producen una transición y otros mantienen a la ambulancia en el

estado actual. Por lo tanto, todas las clases son válidas y no se identifican clases inválidas; la diferencia entre ellas se refleja en la salida esperada de cada caso de prueba.

Condiciones de Entrada (Estado)	**Nro.**	**Clases Válidas**	**Nro.**	**Clases Inválidas**
Disponible	1.1	Traslado	-	-
	1.2	AtencionDomicilio		
	1.3	Mantenimiento		
	1.4	RetornoClinica		
AtendiendoA Domicilio	2.1	RetornoClinica	-	-
	2.2	Traslado OR AtencionDomicilio OR Mantenimiento		
EnTaller	3.1	Mantenimiento	-	-
	3.2	Traslado OR AtencionDomicilio OR RetornoClinica		
RegresandoDel Taller	4.1	RetornoClinica	-	-
	4.2	Traslado OR AtencionDomicilio OR Mantenimiento		
Regresando ClinicaSin Paciente	5.1	Traslado	-	-
	5.2	AtencionDomicilio		
	5.3	RetornoClinica		
	5.4	Mantenimiento		
Trasladando PacienteClinica	6.1	RetornoClinica	-	-

	6.2	Traslado OR AtencionDomicilio OR Mantenimiento		

Casos de Prueba

ID Caso de Prueba	Tipo de Clase	Valores Entrada	Salida Esperada	Clases de Prueba Cubiertas	Esc.
1	Correcta	DisponibleState - SolicitudDeTraslado	TrasladandoPaciente ClinicaState	1.1	2
2	Correcta	DisponibleState - SolicitudDeAtencion Domicilio	AtendiendoEnDomicilio State	1.2	2
3	Correcta	DisponibleState - Solicitud Mantenimiento	EnTallerState	1.3	2
4	Correcta	DisponibleState - RetornoClinica	DisponibleState	1.4	2
5	Correcta	AtendiendoEn DomicilioState- RetornoClinica	RegresandoClinicaSin PacienteState	2.1	2
6	Correcta	AtendiendoEn DomicilioState- SolicitudDeTraslado	AtendiendoEn DomicilioState	2.2	2
7	Correcta	AtendiendoEn DomicilioState- Solicitud Mantenimiento	AtendiendoEn DomicilioState	2.2	2
8	Correcta	AtendiendoEn DomicilioState- SolicitudDeAtencion Domicilio	AtendiendoEn DomicilioState	2.2	2

9	Correcta	EnTallerState-Solicitud Mantenimiento	RegresandoDel TallerState	3.1	2
10	Correcta	EnTallerState-RetornoClinica	EnTallerState	3.2	2
11	Correcta	EnTallerState-SolicitudDeAtencion Domicilio	EnTallerState	3.2	2
12	Correcta	EnTallerState-SolicitudDeTraslado	EnTallerState	3.2	2
13	Correcta	RegresandoDelTaller State-RetornoClinica	DisponibleState	4.1	2
14	Correcta	RegresandoDelTaller State-SolicitudDeTraslado	RegresandoDel TallerState	4.2	2
15	Correcta	RegresandoDelTaller State-SolicitudDeAtencion Domicilio	RegresandoDel TallerState	4.2	2
16	Correcta	RegresandoDelTaller State- Solicitud Mantenimiento	RegresandoDel TallerState	4.2	2
17	Correcta	RegresandoClinica SinPacienteState-SolicitudDeTraslado	TrasladandoPaciente ClinicaState	5.1	2
18	Correcta	RegresandoClinica SinPacienteState-SolicitudDeAtencion Domicilio	AtendiendoEn DomicilioState	5.2	2
19	Correcta	RegresandoClinica SinPacienteState-RetornoClinica	DisponibleState	5.3	2

20	Correcta	RegresandoClinica SinPacienteState-Solicitud Mantenimiento	RegresandoClinica SinPacienteState	5.4	2
21	Correcta	Trasladando PacienteClinicaState - RetornoClinica	DisponibleState	6.1	2
22	Correcta	Trasladando PacienteClinicaState - SolicitudDeTraslado	TrasladandoPaciente ClinicaState	6.2	2
23	Correcta	Trasladando PacienteClinicaState - SolicitudDeAtencion Domicilio	TrasladandoPaciente ClinicaState	6.2	2
24	Correcta	Trasladando PacienteClinicaState - SolicitudMantenimie nto	TrasladandoPaciente ClinicaState	6.2	2

Resultados de Pruebas

Test	ID Caso de Prueba asociadas	Resultado
disponibleDebePasarATrasladandoPacienteClinica	1	OK
disponibleDebePasarAAtendiendoEnDomicilio	2	OK
disponibleDebePasarAEnTaller	3	OK
disponibleDebeSeguirDisponible	4	OK
debePasarDeAtendiendoDomicilioARetornoAClinica	5	OK
atendiendoADomiclioDebeSeguirAtendiendo	6	OK

atendiendoADomicilioDebeRechazarMantenimiento	7	OK
atendiendoADomicilioDebeRechazarAtencionDomicilio	8	OK
enTallerDebeTerminarMantenimiento	9	OK
enTallerDebeNoRetornarALaClinica	10	OK
enTallerDebeNoAtenderDomicilio	11	OK
enTallerDebeNoTrasladarPaciente	12	OK
RegresandoTallerDebePasarADisponible	13	OK
RegresandoTallerNoDebePasarATrasladandoPaciente Clinica	14	OK
RegresandoTallerNoDebePasarAAtencionADomicilio	15	OK
RegresandoTallerNoDebePasarAMantenimiento	16	OK
regresandoAClinicaDebePasarATrasladandoPaciente	17	OK
regresandoAClinicaDebePasarAAtendiendoDomicilio	18	OK
regresandoAClinicaDebePasarADisponible	19	OK
regresandoAClinicaNoDebePasarAMantenimiento	20	OK
traslandoPacienteDebePasarADisponible	21	Fallo
traslandoPacienteDebeSeguirTrasladando	22	OK
traslandoPacienteNoDebeAtenderDomicilio	23	OK
traslandoPacienteNoDebeIrAMantenimiento	24	OK

Fallo: traslandoPacienteDebePasarADisponible()

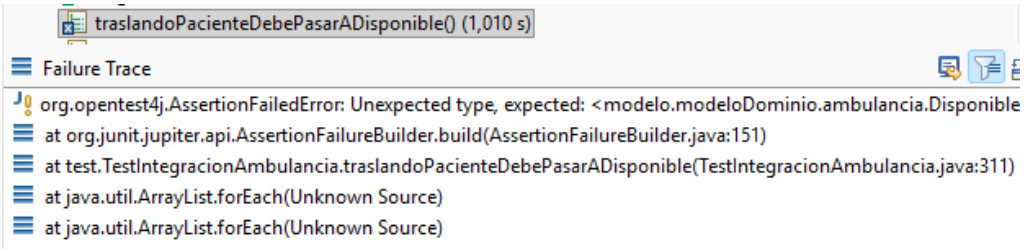
```
@Test
void traslandoPacienteDebePasarADisponible() {
    Ambulancia ambulancia = new Ambulancia();

    ambulancia.setState(new TrasladandoPacienteClinicaState(ambulancia));

    ambulancia.getEstadoActual().RetornoClinica();

    assertInstanceOf(
            DisponibleState.class,
            ambulancia.getEstadoActual()
        );
}
```

El test corresponde a la clase 6.1: estado TrasladandoPacienteClinica y evento RetornoClinica. Partiendo de una ambulancia ubicada en el estado TrasladandoPacienteClinicaState, se invoca RetornoClinica() sobre el estado actual y se verifica, mediante assertInstanceOf, que la ambulancia haya pasado al estado DisponibleState. Una vez que la ambulancia llega a la clínica con el paciente, el traslado finaliza y la ambulancia debe quedar libre para atender nuevas solicitudes.



El test falla en el assertInstanceOf: JUnit informa que se esperaba una instancia de DisponibleState, pero el estado actual de la ambulancia es RegresandoDelTallerState. En este caso no se lanza ninguna excepción durante la ejecución: la transición se realiza, pero hacia un estado incorrecto. La falla corresponde entonces a una diferencia entre el resultado esperado y el obtenido.

- **Causa:** Cuando se invoca a RetornoClinica en una ambulancia cuyo estado es TrasladandoPacienteClinicaState, el estado nuevo y esperado debería ser DisponibleState pero en el código encontramos que la ambulancia pasa al estado RegresandoDelTallerState.
- **Solución Propuesta:** Que al invocar al método ya nombrado el estado de la ambulancia pase a ser DisponibleState, para esto deberíamos modificar la línea 56 de la clase TrasladandoPacienteClinicaState por: this.ambulancia.setEstado(new StateDisponible(this.ambulancia)).

Suite 3: AmbulanciaConcurrenciaTest

Casos de Prueba

ID Caso de Prueba	Tipo de Clase	Valores Entrada	Salida Esperada	Clases de Prueba Cubiertas	Esc.
1	Correcta	Ambulancia en TrasladandoPacientClinicaState con ocupado = true	Ambos asociados esperan y acceden de a uno	-	4

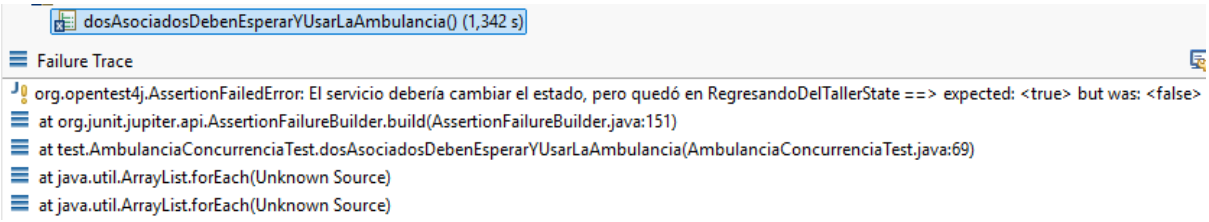
Resultados de Pruebas

Test	ID Caso de Prueba asociadas	Resultado

dosAsociadosDebenEsperarYUsarLaAmbulancia	1	Fallo

Fallo: dosAsociadosDebenEsperarYUsarLaAmbulancia

La ambulancia se encuentra en estado TrasladandoPacienteClinica y ocupada, simulando que está prestando otro servicio. Se lanzan dos hilos en forma simultánea: el asociado A solicita un traslado y el asociado B una atención a domicilio. Mediante un observador y un CountDownLatch, el test verifica primero que ambos asociados queden en espera. Luego libera la ambulancia con regresarSinPaciente() y verifica que exactamente uno de los asociados obtenga el servicio, es decir, que la ambulancia pase a TrasladandoPacienteClinica o a AtendiendoEnDomicilio. Finalmente, la libera nuevamente y verifica que el segundo asociado también sea atendido.



La primera parte del test se cumple: ambos asociados quedan en espera mientras la ambulancia está ocupada y, al liberarse, uno de ellos es despertado y completa su solicitud. Sin embargo, el test falla en la verificación del estado posterior, ya que la ambulancia no pasó a ninguno de los estados de servicio, sino que quedó en RegresandoDelTallerState.

- Corrección propuesta: reemplazar new RegresandoDelTallerState(this.ambulancia) por new DisponibleState(this.ambulancia) en la línea 56 de TrasladandoPacienteClinicaState.

Suite 4: MantenimientoConcurrenciaTest

Casos de Prueba

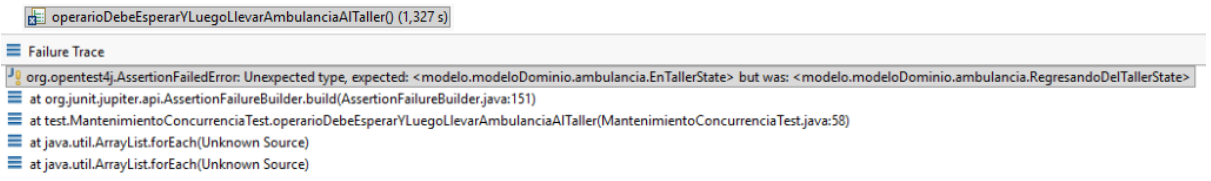
ID Caso de Prueba	Tipo de Clase	Valores Entrada	Salida Esperada	Clases de Prueba Cubiertas	Esc.
1	Correcta	Ambulancia en TrasladandoPacienteClinicaState y ocupada - solicitarMantenimiento	El operario espera y luego pasa a EnTallerState	-	5

Resultados de Pruebas

Test	ID Caso de Prueba asociadas	Resultado
operarioDebeEsperarYLuegoLlevarAmbulanciaAlTaller	1	Fallo

Fallo: operarioDebeEsperarYLuegoLlevarAmbulanciaAlTaller()

La ambulancia se encuentra en estado TrasladandoPacienteClinica y ocupada. En un hilo independiente, un operario solicita mantenimiento. Mediante un observador y un CountDownLatch, el test verifica primero que el operario quede en espera mientras la ambulancia está ocupada. Luego libera la ambulancia con regresarSinPaciente() y verifica que el operario obtenga la ambulancia, es decir, que pase a EnTallerState y quede marcada como ocupada durante el mantenimiento. Por último, simula la salida del taller (volviendoDelTaller() y regresarSinPaciente()) y verifica que la ambulancia vuelva a DisponibleState y quede libre.



La primera parte del test se cumple: el operario espera mientras la ambulancia está ocupada y, al liberarse, es despertado y completa su solicitud. Sin embargo, el test falla en el assertInstanceOf de la línea 58: se esperaba que la ambulancia estuviera en EnTallerState, pero se encuentra en RegresandoDelTallerState.

- **Corrección propuesta:** reemplazar new RegresandoDelTallerState(this.ambulancia) por new DisponibleState(this.ambulancia) en la línea 56 de TrasladandoPacienteClinicaState.

Suite 5: RetornoAutomaticoTest

Casos de Prueba

ID Caso de Prueba	Tipo de Clase	Valores Entrada / Evento	Salida Esperada	Clases de Prueba Cubiertas	Esc.
1	Correcta	Inicio del hilo RetornoAutomatico y espera a que se cumpla el intervalo de 10 s.	Se invoca al menos una vez estado.RetornoClinica() a través de ambulancia.retornoAutomatico().	-	6
2	Correcta	Inicio del hilo RetornoAutomatico con la simulación ya finalizada.	El hilo finaliza sin invocar nunca estado.RetornoClinica().	-	7
3	Correcta	Inicio del hilo, espera hasta que entre en TIMED_WAITING e invocación a hilo.interrupt().	El hilo captura InterruptedException, finaliza limpiamente (!hilo.isAlive()) y no invoca estado.RetornoClinica().	-	6
4	Correcta	Cumplimiento del primer retorno y llamado a setSimulacionActiva(false) durante el segundo ciclo de espera (TIMED_WAITING).	El hilo se despierta, detecta que la simulación finalizó, sale del bucle y registra una única invocación a estado.RetornoClinica().	-	6
5	Correcta	Ejecución concurrente del hilo RetornoAutomatico y un hilo Asociado que invoca ambulancia.trasladarALaClinica(asociado).	Ambas operaciones se ejecutan en exclusión mutua sobre el monitor de Ambulancia (maximoSimultaneas == 1).	-	6

Resultados de Pruebas

Todos los tests de esta suite siguen la estructura de preparar, ejecutar y verificar:

1. **Preparación:** se crea una Ambulancia real y se le inyecta un mock de State mediante setState(estado) para aislar la prueba de los estados concretos. Según el caso, la simulación se deja activa (escenario 6) o se finaliza previamente con setSimulacionActiva(false) (escenario 7), y se instancia el hilo Thread pasándole new RetornoAutomatico(ambulancia).
2. **Ejecución:** se inicia el hilo con start() y se aplica el estímulo correspondiente a cada caso: esperar a que transcurran los 10 segundos del INTERVALO_RETORNO, interrumpir el hilo con interrupt() mientras está en espera (TIMED_WAITING), apagar la simulación luego del primer retorno, o lanzar en paralelo un segundo hilo con un Asociado pidiendo traslado a la clínica.
3. **Verificación:** se comprueba mediante verify de Mockito si la ambulancia delegó (o no) el llamado a estado.RetornoClinica(), se verifica con assertFalse(hilo.isAlive()) que el hilo haya terminado correctamente sin quedar colgado, y en el caso de concurrencia se controla mediante un contador atómico que nunca se hayan solapado dos operaciones al mismo tiempo adentro del monitor.

Test	ID Caso de Prueba asociadas	Resultado
debeEjecutarRetornoAlCumplirseElIntervalo	1	Fallo
noDebeEjecutarRetornoSiLaSimulacionYaFinalizo	2	OK
debeFinalizarCuandoSeInterrumpe	3	Fallo
noDebeRetornarDespuesDeFinalizarLaSimulacion	4	Fallo
retornoYSolicitudDebenEstarSincronizados	5	Fallo

Fallos: debeEjecutarRetornoAlCumplirseElIntervalo, debeFinalizarCuandoSeInterrumpe, noDebeRetornarDespuesDeFinalizarLaSimulacion y retornoYSolicitudDebenEstarSincronizados

```
25⊖         while (!ambulancia.isSimulacionActiva()) {
```

Los cuatro tests del Escenario 6 (simulación activa) fallan con AssertionFailedError por un único defecto en la línea 25 de RetornoAutomatico.run(): la condición del bucle está negada (while (!ambulancia.isSimulacionActiva())). Al estar la simulación activa (true), la condición da false de entrada, por lo que el hilo nunca entra al while y termina inmediatamente sin

pausarse en el Thread.sleep() ni invocar nunca a ambulancia.retornoAutomatico(this). Peor aún, esto vuelve inalcanzable al retorno automático: solo entra al bucle cuando la simulación está inactiva (caso 2), pero al salir del sleep se topa con el if (!ambulancia.isSimulacionActiva()) break; de la línea 34 y corta sin hacer nada.

- **Causa:** la condición del bucle en RetornoAutomatico.run (línea 25) está invertida (!ambulancia.isSimulacionActiva() en lugar de ambulancia.isSimulacionActiva()).
- **Corrección propuesta:** eliminar el operador ! en la línea 25 de RetornoAutomatico.java, dejando while (ambulancia.isSimulacionActiva()).

Conclusiones

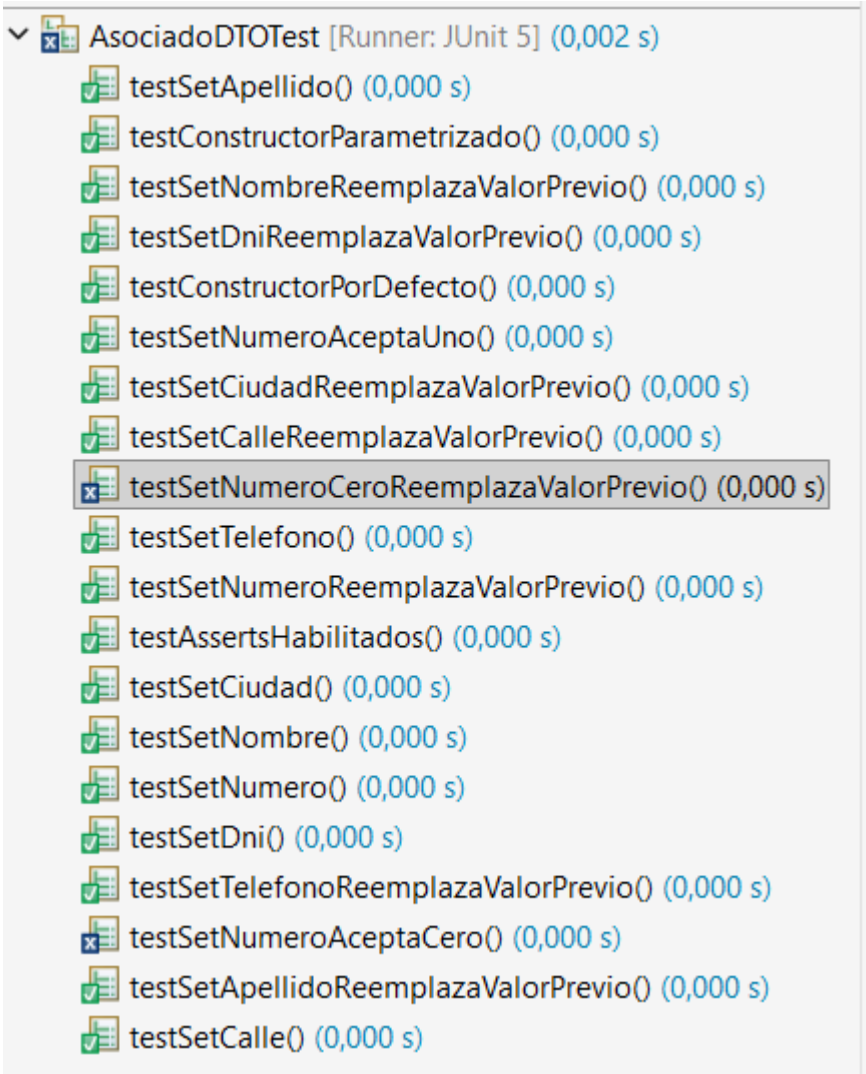
Se diseñaron y ejecutaron 5 suites complementarias (TestAmbulancia, TestIntegracionAmbulancia, AmbulanciaConcurrenciaTest, MantenimientoConcurrenciaTest y RetornoAutomaticoTest) que evalúan de forma incremental la colaboración entre el contexto Ambulancia, la jerarquía del patrón State y el hilo de RetornoAutomatico a lo largo de los 7 escenarios definidos. El uso de objetos simulados con Mockito (State, Asociado y Operario) junto con primitivas de sincronización (CountDownLatch y Semaphore) permitió aislar responsabilidades y controlar la ejecución concurrente de los hilos de manera determinística.
El contraste entre las suites permitió acotar con precisión el origen de cada defecto: mientras que las 7 pruebas con mocks de TestAmbulancia resultaron exitosas (confirmando que Ambulancia delega correctamente sus operaciones y gestiona el flag ocupado), la integración con las clases reales expuso dos defectos críticos que comprometían el funcionamiento del subsistema:

- **Transición inválida en TrasladandoPacienteClinicaState.RetornoClinica()** (línea 56): Al retornar a la clínica, el estado asigna erróneamente RegresandoDelTallerState en lugar de DisponibleState. Este defecto no solo hizo fallar la prueba puntual en TestIntegracionAmbulancia, sino que provocó un fallo en cascada en AmbulanciaConcurrenciaTest y MantenimientoConcurrenciaTest: cuando los hilos de asociados u operarios se despiertan del wait(), encuentran a la ambulancia en RegresandoDelTallerState y sus solicitudes son rechazadas a pesar de haberse liberado el recurso.
- **Condición de bucle invertida en RetornoAutomatico.run()** (línea 25): El encabezado while (!ambulancia.isSimulacionActiva()) impide que el hilo ingrese al bucle cuando la simulación está activa (haciendo fallar los 4 casos del escenario 6 en RetornoAutomaticoTest) y vuelve inalcanzable la línea ambulancia.retornoAutomatico(this).

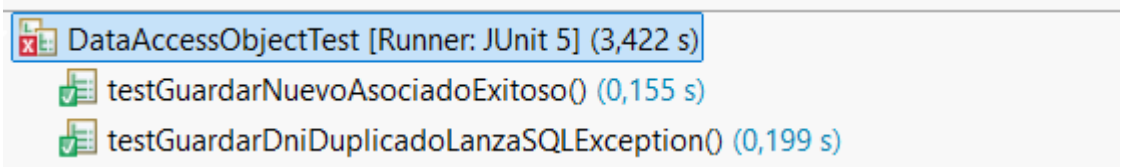
En conjunto, estos resultados demuestran el valor de combinar pruebas aisladas mediante mocks con pruebas de integración y concurrencia: los mocks permitieron descartar fallas de delegación en Ambulancia, mientras que las pruebas integradas revelaron cómo un único error de transición en un estado concreto o una condición booleana invertida terminan bloqueando la atención concurrente de pacientes y el retorno automático del sistema.

4. Apéndice

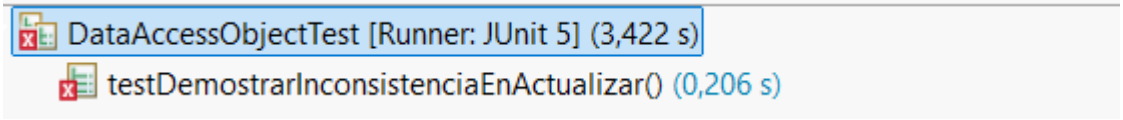
4.1 Resultados AsociadoDTO

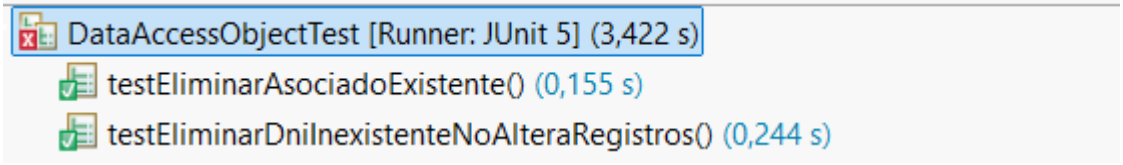


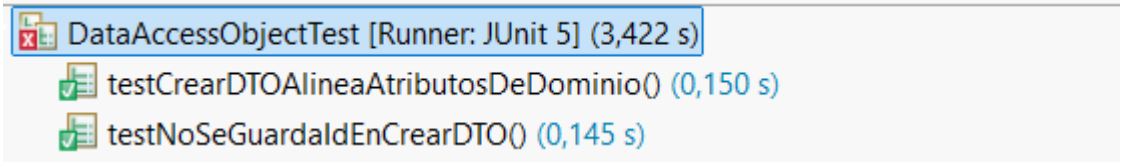
4.2 Resultados DataAccessObject







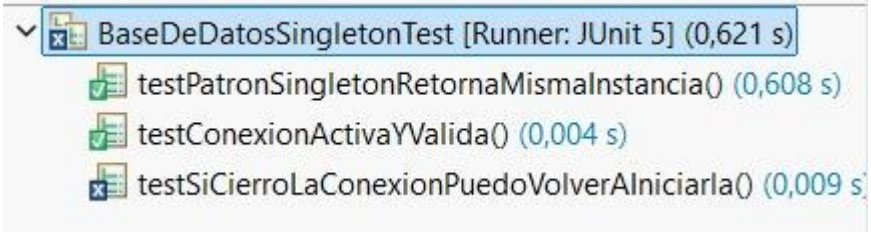




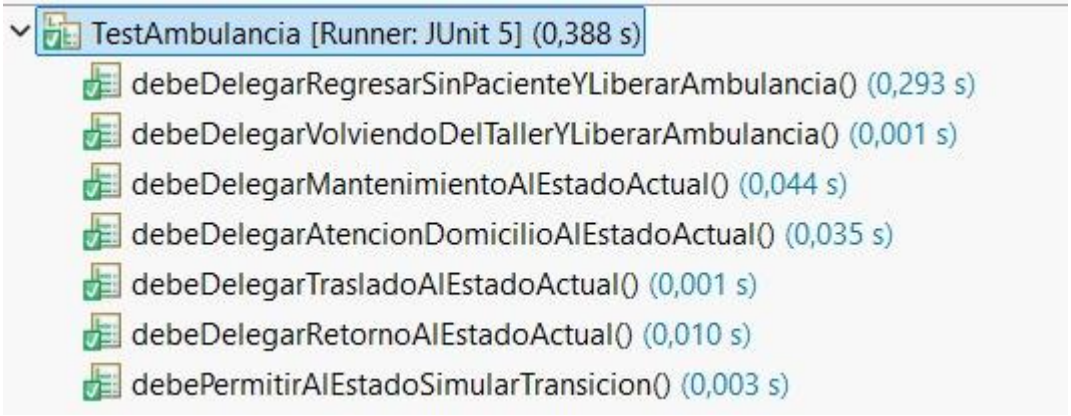
4.3 Resultados BaseDeDatos



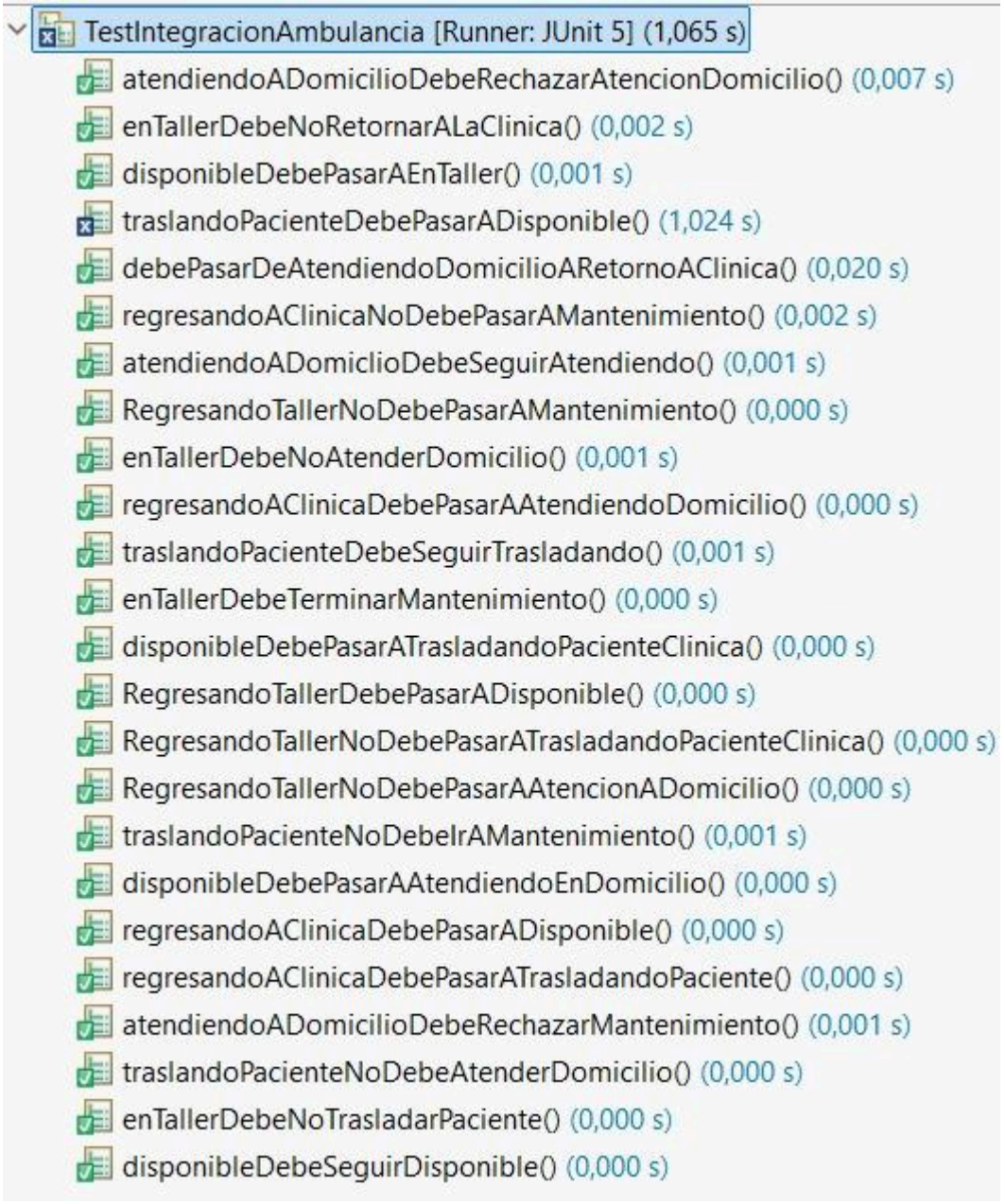
4.4 Resultados BaseDeDatosSingleton



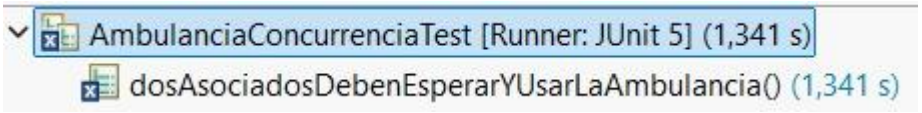
4.5 Resultados Ambulancia



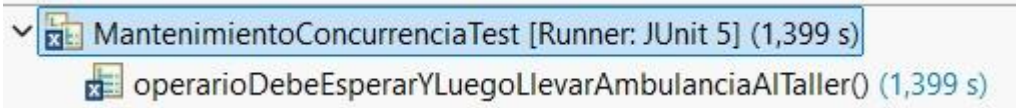
4.6 Resultados IntegracionAmbulancia



4.7 Resultados ConcurrenciaAmbulancia



4.8 Resultados MantenimientoConcurrencia



4.9 Resultados RetornoAutomatico

